

Homework #2 – Problem #2

Algorithm

Algorithm 1 Deep Q-Network

```
1: Initialize Q-network weights  $\theta$ 
2: Initialize the replay buffer  $\mathcal{B}$  with capacity  $D$ .
3: Initialize target network  $\hat{Q}_{\theta'}$ , weights  $\theta' = \theta$ 
4: for episode=1,...,  $M$  do
5:   for  $t = 1, \dots, T$  do
6:     Based on the current state  $s_t$ , choose action  $a_t$  according to  $\epsilon$ -Greedy policy.
7:     Take action  $a_t$ , collect a tuple  $(s_t, a_t, r_t, s_{t+1})$  and add to the replay buffer  $\mathcal{B}$ .
8:     Randomly sample a mini-batch of tuples  $\mathcal{S} = \{(s_i, a_i, r_i, s_{i+1})\}$  from the replay buffer  $\mathcal{B}$ .
9:     for each tuple  $(s_i, a_i, r_i, s_{i+1})$  in the batch  $\mathcal{S}$  do
10:      Compute  $y_i$ :
11:      if episode terminates after step  $i$  then
12:         $y_i = r_i$ 
13:      else
14:         $y_i = r_i + \gamma \max_{a'} \hat{Q}_{\theta'}(s_{i+1}, a')$  using the target network with weights  $\theta'$ 
15:      end if
16:    end for
17:    Update  $\theta$  by taking an optimization step on the loss  $L(\theta) = \sum_i (Q_{\theta}(s_i, a_i) - y_i)^2$ 
18:    Update  $\theta'$  after every  $N$  optimization steps.
19:  end for
20: end for
```

The implementation of this algorithm is like problem 1. In Addition to that, I used convolutional layers in the neural networks of both the policy and target networks (they have the same neural network architecture).

Best Run Results

In this run I used my code saved in file: hw2_p2_oct31.py to train the neural networks. This code was run for 2900 episodes, until there was an interruption due to a technical issue (I don't recall if my PC got restarted or the task was killed by the server).

Afterwards, I loaded the numpy arrays of Qmax, episode rewards, and the models (.pth files) of the policy network and target network that were saved at episode 2900, and continued training the model, using the code in file: hw2_p2_nov2_continue.py until episode 3050 since the reward of the model has converged. In the folder you will find the saved policy model, Qmax numpy array, and episode rewards numpy array at episode 3050.

Afterwards, I used the saved trained policy model, and rolled 500 episodes and saved their rewards and plotted a histogram.

Please note that achieving an average episode reward of around ~ 1200 is a good target since the paper by Mnih et al. (2015), have showed that DQN can achieve an average reward of ~ 2311 (reference: <https://www.nature.com/articles/nature14236>) however this comes after

several experiments and an optimized implementation. So, achieving an average of 1200 within less than 10 days is pretty acceptable. Note that during the training the average of the last 100 episodes has passed the score of 1250 several times and it kept fluctuating around ~ 1000 .

Not that figure 1 shows the rewards of the episodes till episode 3050, but if we zoom in to the first 500 episodes (in Figure 2), we find that the model already started learning and reached an average of about 1000 episodes around episode 500, which shows fast learning relative to the complexity of the game. And Figures 3 and 4 show the average Qmax values of the training episodes (the latter shows the values only for the first 500 episodes).

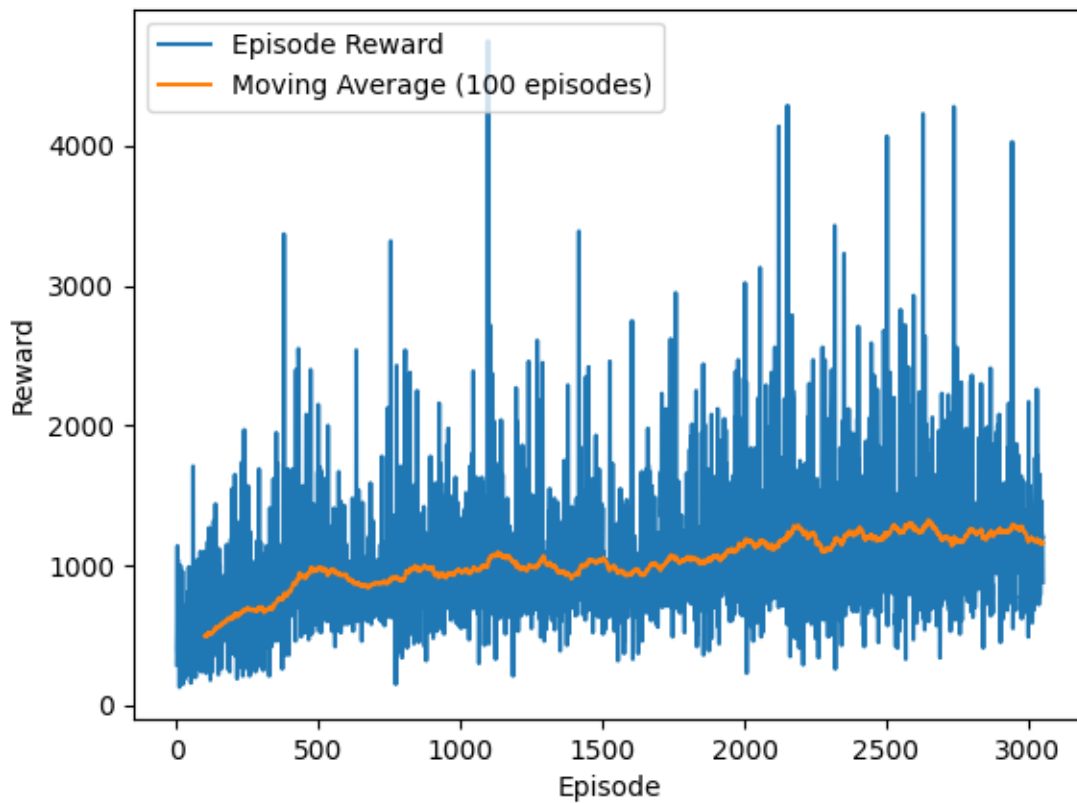


Figure 1: Episode Rewards plot for final run

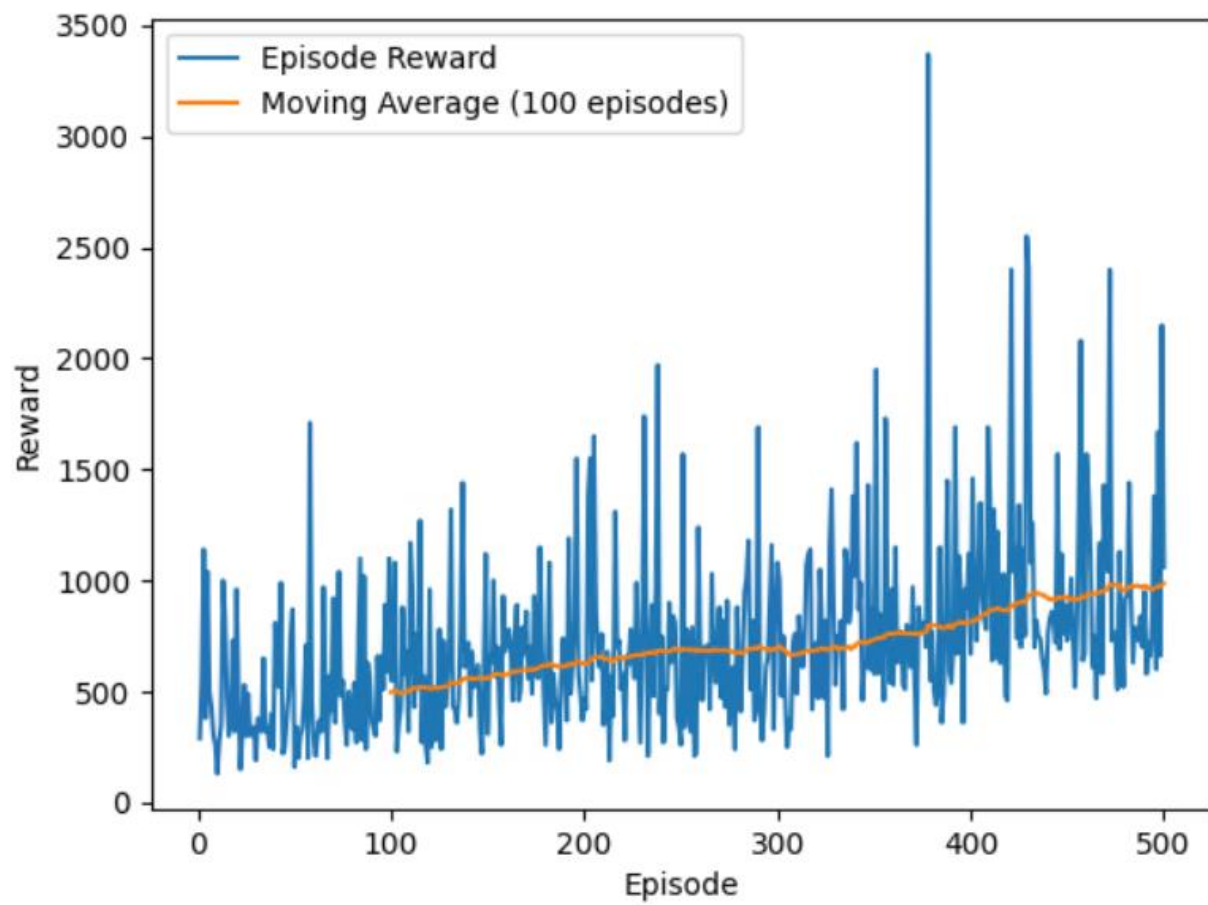


Figure 2: Episode rewards for only the first 500 episodes

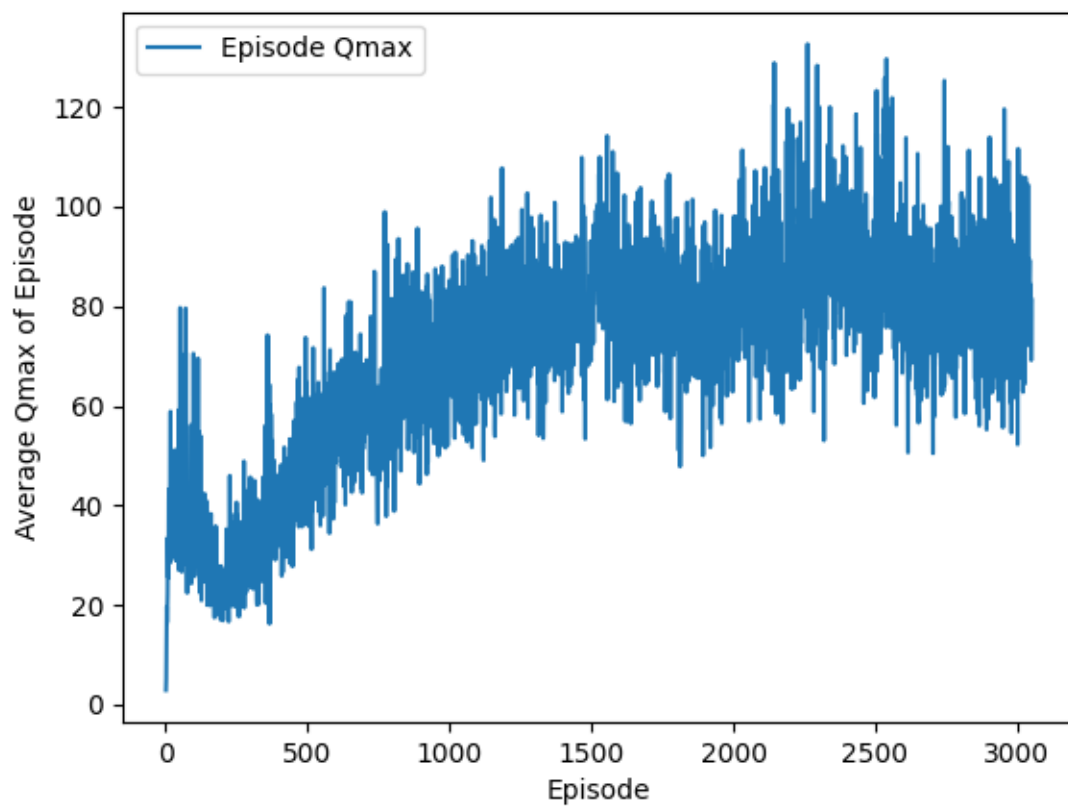


Figure 3: Average Q_{max} plot during training episodes for final run

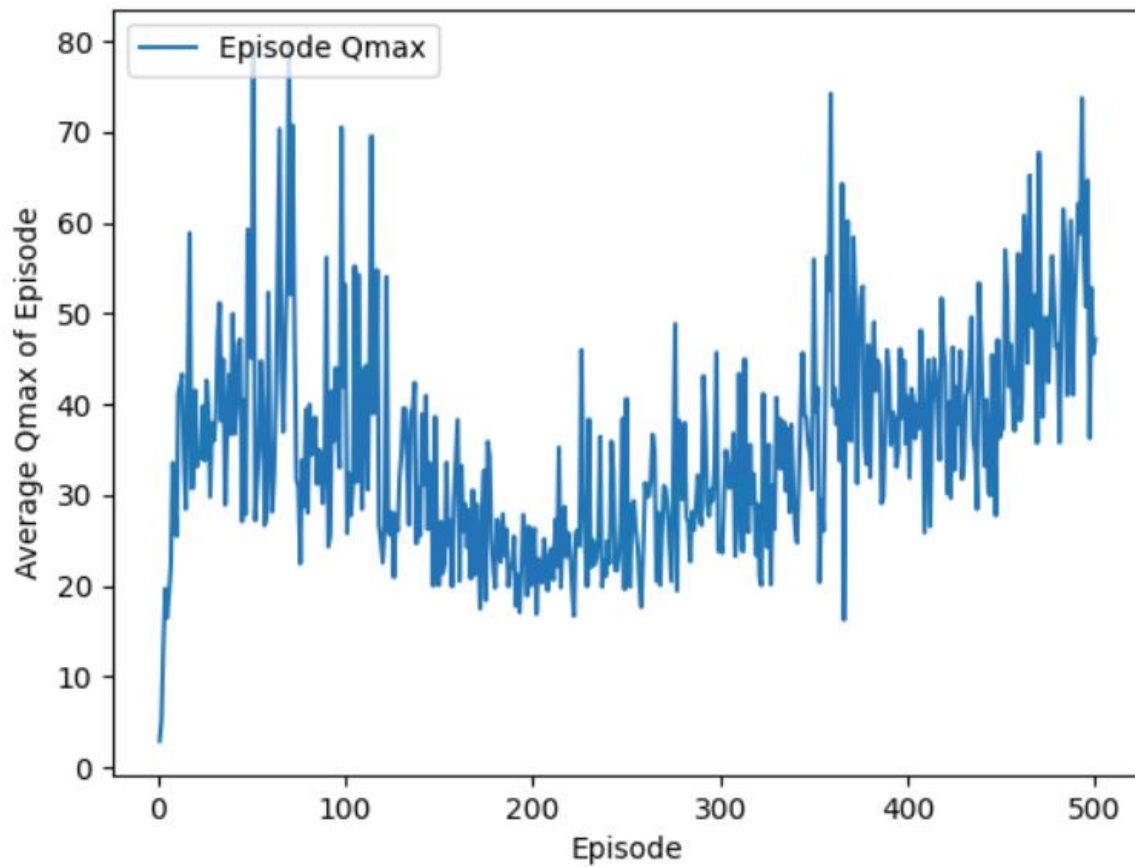


Figure 4: Average Q_{max} plot during the first 500 training episodes

Then 500 episodes are rolled using code: P2-Run1-PlottingSample.ipynb , and a histogram is plotted in Figure 5 that shows the distribution of rewards of these 500 episodes that have an average of 1149 and a standard deviation of ~ 383 . The distribution looks like a skewed normal distribution.

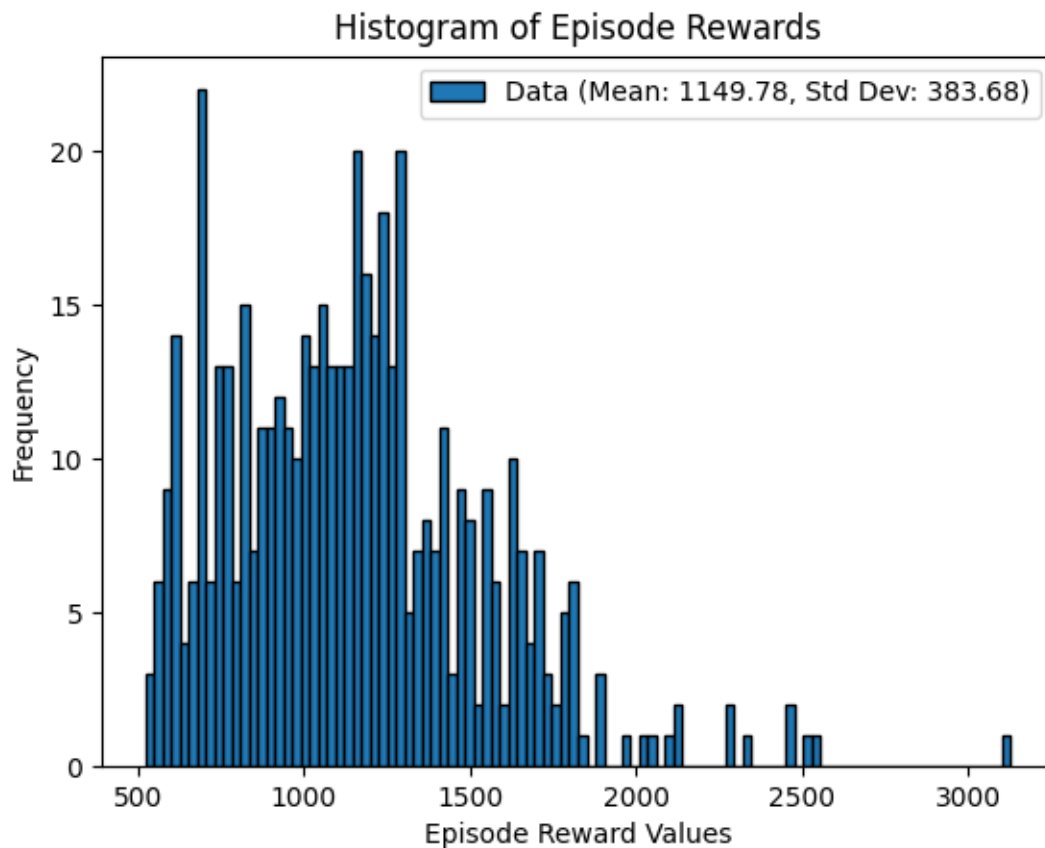
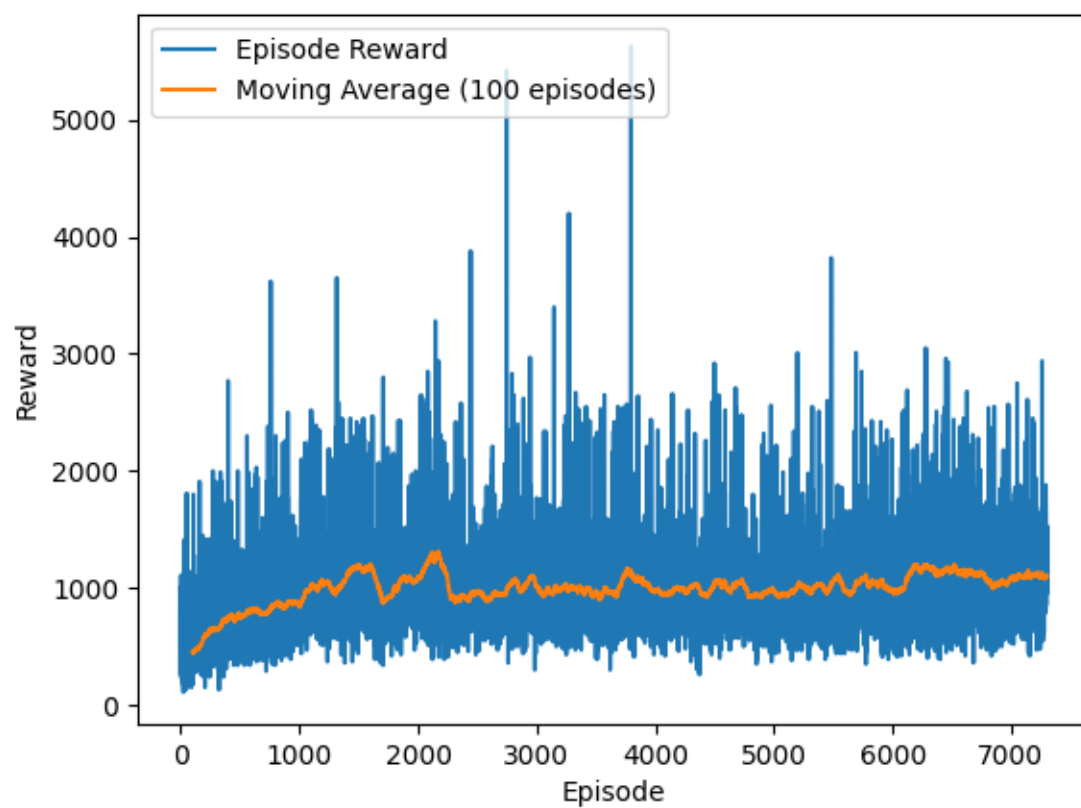


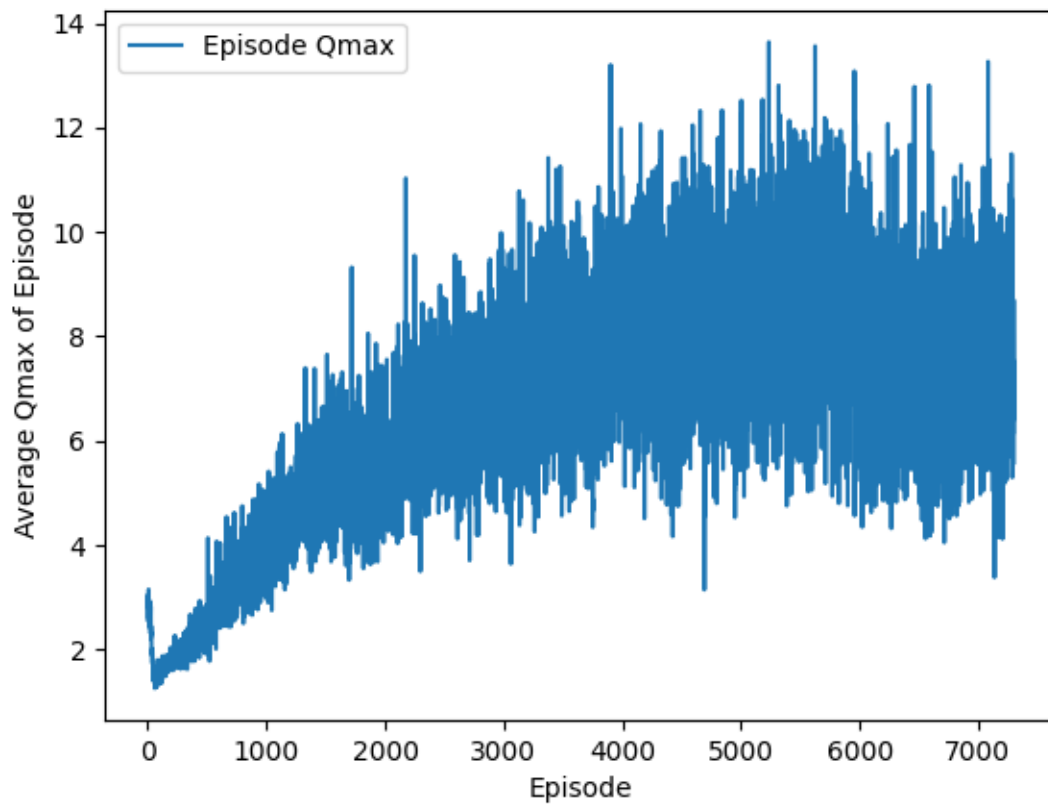
Figure 5: Histogram of Episode Rewards of a sample of 500 episodes using trained model

Other Runs

Run 4

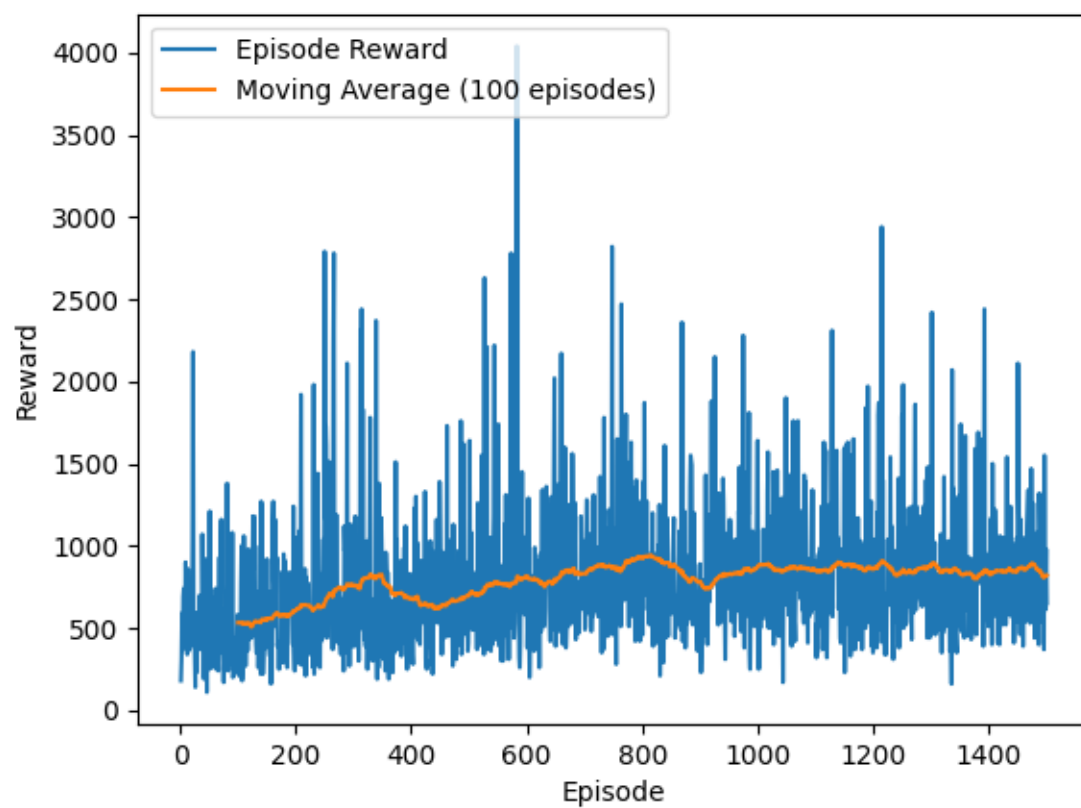
The results of this run are in the folder of run 4. This run has some different hyperparameters and a different neural network architecture. The computation time of one episode for this run is lower than the best run, however it also converges around reward ~ 1100 . Also, the values of Qmax seem lower, although it does have a learning trend. (This could be due to a typo in saving the Q_value in this code, however there was no time to correct it as it was already running). The rewards of the training episodes increase fast during the first 500 episodes, and then shows some convergence with some fluctuation between scores of 900 and 1200 rewards. Also note that in this run, the target model updates its weights once after each episode and not after every optimization step.

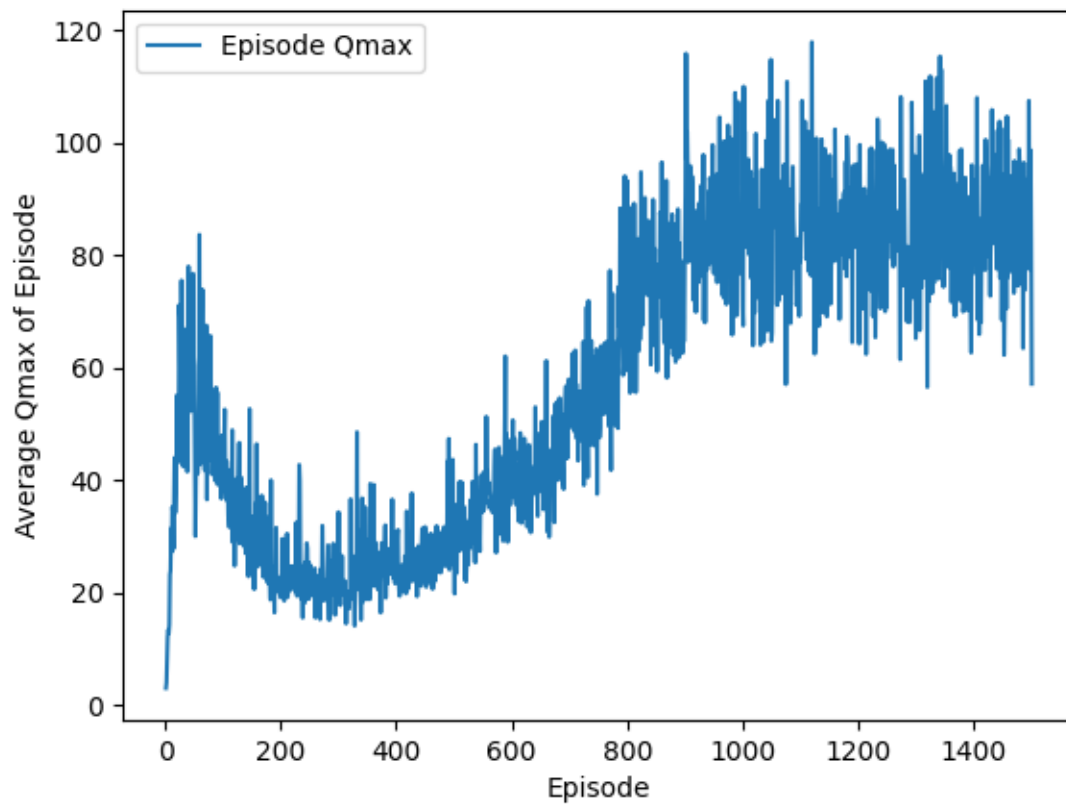




Run 2

This model also shows learning. It was interrupted at episode 900 due to an autokill of the kernel, and the training proceeded until episode 1500. It also shows the most significant learning phase around episodes 1-500, and then it converges. This model has the same hyperparameters and neural network architecture as the best run, except that the batch size is 128 instead of 256. Q also shows quick learning, then it decreases but then increases again.





References

- <https://www.youtube.com/watch?v=EjYESbg95x0>
- PyTorch Tutorial for DQN
- <https://www.nature.com/articles/nature14236>